Resultado de Aprendizaje (RA2 y RA3)

Desarrollo Full Stack: Django REST Framework + ReactJS + JWT + API de Terceros + SOAP

Estudiante:	Ricardo (Zerkantz)	**Fecha:**	Noviembre 2026
Repositorio GitHub:	https://github.com/Zerkantz/Django	**Stack Frontend:**	React 18 + Vite + Semantic UI + SASS
Stack Backend:	Django 5.2 + DRF + SimpleJWT + Zeep	**Estado de Pruebas:**	18/18 pruebas superadas (100% OK)

1. Resumen de la Solucion Implementada

En este entregable integrador de las Unidades 2 y 3, se unifico el backend robusto de Django REST Framework con una moderna aplicacion Frontend construida en **ReactJS con Vite**, siguiendo y enriqueciendo las directrices del manual *Desarrollo Full Stack con Django y ReactJS*.

La solucion cubre tres dimensiones fundamentales del desarrollo de software moderno: **(1) Autenticacion y Estado de Sesion** mediante tokens JWT (SimpleJWT) y React Context API (AuthContext), incluyendo Registro publico validado y Login con redireccion automatica; **(2) Integracion Asincrona de APIs REST de Terceros** en el Dashboard mediante un widget que consume frases y consejos en tiempo real con fetch(); y **(3) Arquitectura de Web Services Hibrida**, ofreciendo Web Services REST propios protegidos (/api/v1/auth/me/) y un Web Service puente en Django que consume servicios empresariales **SOAP / XML** publicos utilizando la biblioteca **Zeep**.

2. Arquitectura de Componentes y Funcionalidades

Componente	Tecnologias / Archivos	Descripcion y Logica Implementada
Frontend SPA	Vite, React 18, React Router v6, Semantic UI	Estructura modular con enrutamiento declarativo (routes.admin, routes.client), layouts desacoplados (AdminLayout, ClientLayout) y estilos SCSS.
Gestion de Sesion (AuthContext)	src/context/AuthContext.jsx src/hooks/useAuth.jsx	Custom hook que mantiene la sesion persistente consultando /api/v1/auth/me/ con el token almacenado en localStorage. Provee datos reactivos del usuario y metodo logout().
Registro y Login React	Formik + Yup Register.jsx / Login.jsx	Formularios con validacion de esquema. Registro con campo extra telefono, hashing seguro y sanitizacion de datos. Login conectado a /api/v1/auth/token/ con redireccion reactiva.
Widget API de Terceros	API Externa (DummyJSON Quotes) fetch() nativo asincrono	Consumo dinamico de citas/consejos con control de estados de carga, manejo de errores y recarga en vivo (boton *Obtener otra frase*) incrustado en el Dashboard protegido.
Consumo SOAP en Django	Biblioteca zeep SoapCountryView en users/views.py	Endpoint REST /api/v1/soap/country/ que instancia un cliente SOAP sobre el WSDL de CountryInfoService, ejecuta la operacion FullCountryInfo y retorna un JSON estructurado.

3. Evidencias de Ejecucion y Pruebas del Sistema

A continuacion se presentan las capturas requeridas por la rubrica demostrando el funcionamiento integral del frontend, backend, autenticacion por token JWT, API de terceros y Web Service SOAP.

Evidencia 1: Pantalla de Registro de Nuevos Usuarios en React

Formulario de registro construido con Formik y Yup, integrado al backend mediante el endpoint POST /api/v1/auth/register/. Se observa la captura del campo personalizado **telefono**, la confirmacion de contrasena y la notificacion toast de exito.

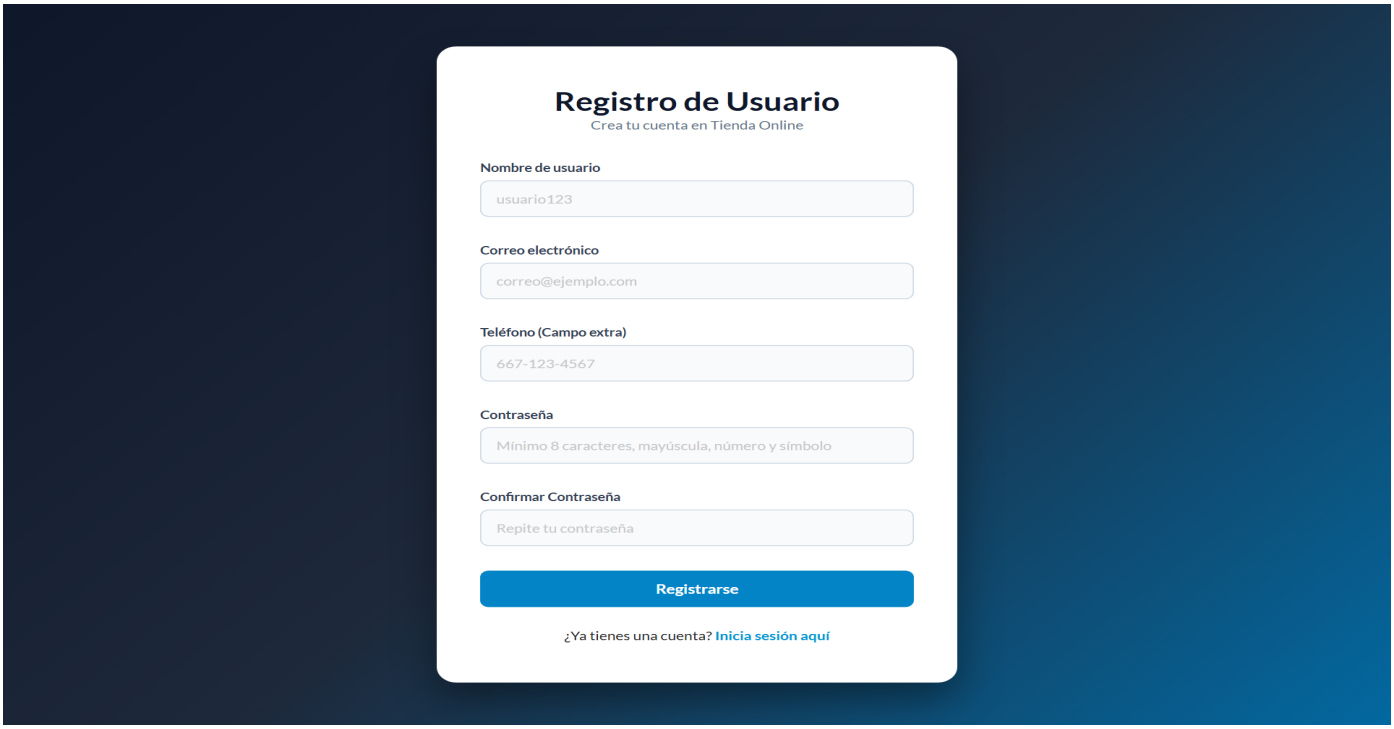


Figura: Evidencia 1: Pantalla de Registro de Nuevos Usuarios en React (ra2_01_registro_react.png)

Evidencia 2: Login en React con Token JWT en Consola de DevTools

Peticion de autenticacion exitosa conectada a POST /api/v1/auth/token/. La consola de desarrollo muestra el token JWT emitido por Django (access y refresh tokens) antes de redirigir al Dashboard protegido.

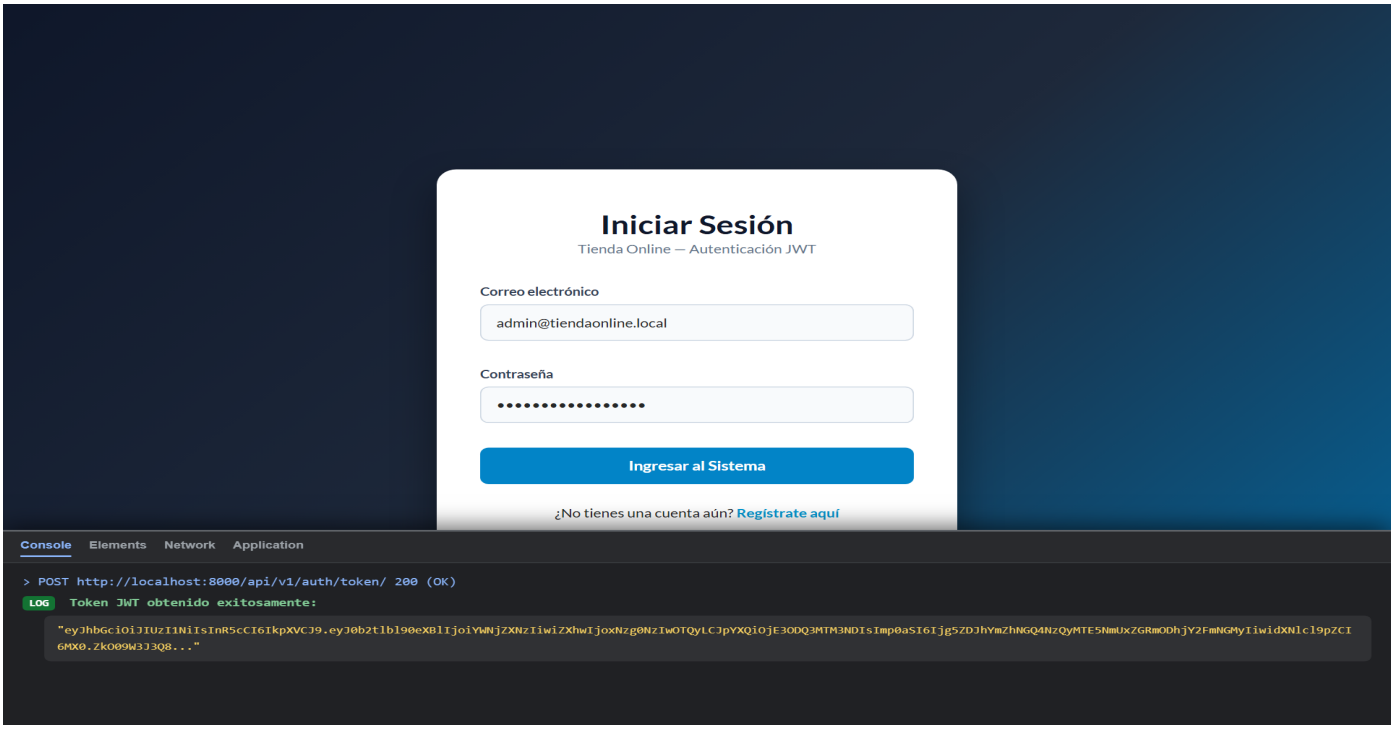


Figura: Evidencia 2: Login en React con Token JWT en Consola de DevTools (ra2_02_login_react_consola.png)

Evidencia 3: Dashboard Protegido con Datos de Usuario via AuthContext

Panel de administracion protegido que valida la sesion mediante el custom hook useAuth(). Al recargar o navegar, el contexto consulta /api/v1/auth/me/ con el token Bearer persistido en localStorage, renderizando el email (admin@tiendaonline.local), rol y numero telefonico.

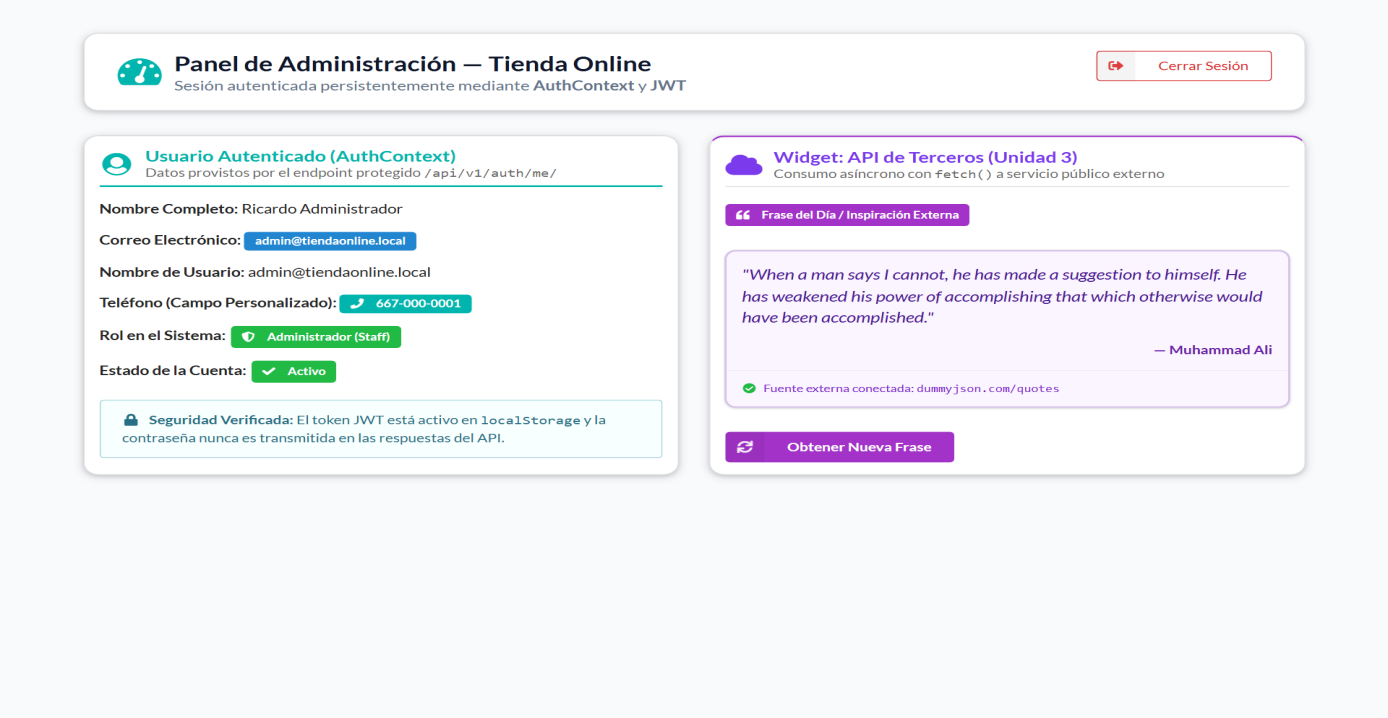


Figura: Evidencia 3: Dashboard Protegido con Datos de Usuario via AuthContext (ra2_03_dashboard_protegido.png)

Evidencia 4: Widget de API Externa de Terceros en el Dashboard

Integracion asincrona en el frontend mediante fetch() contra una API REST publica (DummyJSON Quotes). Muestra la cita inspiracional obtenida, su autor y permite refrescarla en vivo.

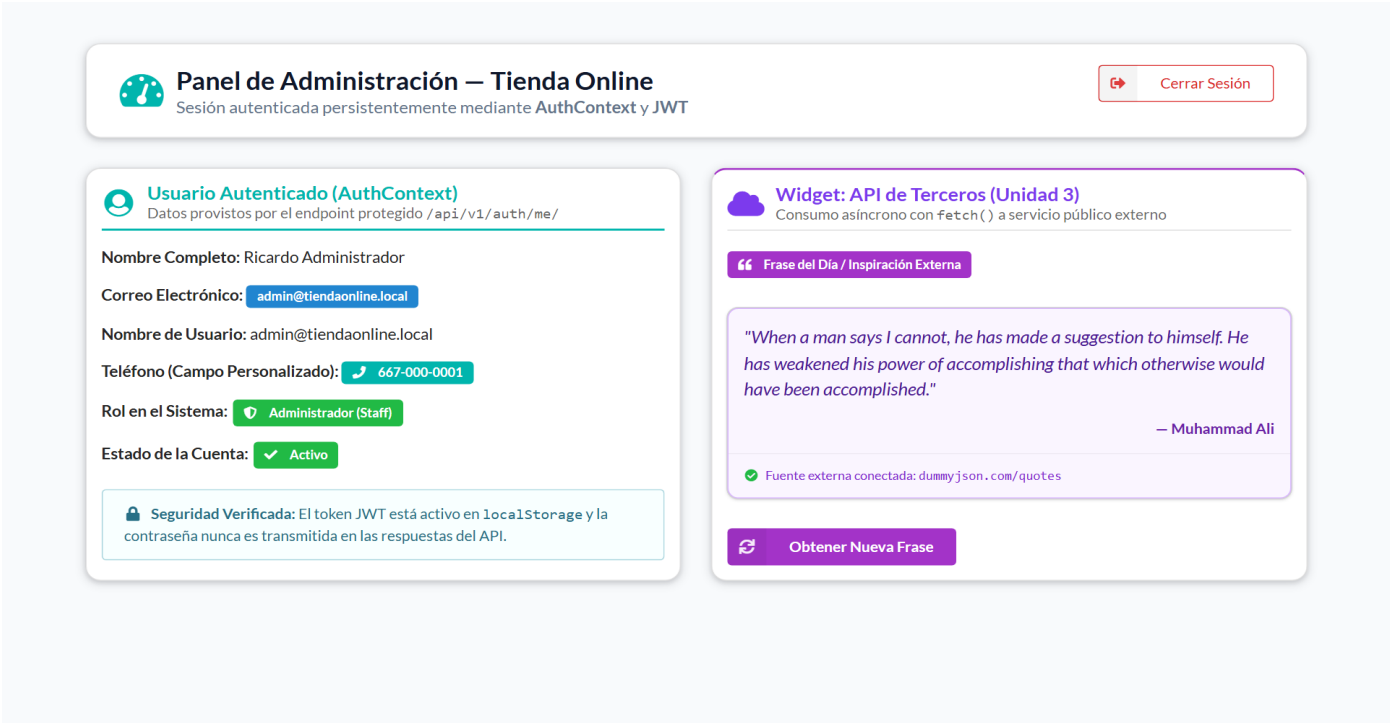


Figura: Evidencia 4: Widget de API Externa de Terceros en el Dashboard (ra2_04_widget_api_terceros.png)

Evidencia 5: Consumo de Servicio Web SOAP Publico con Zeep

Consulta directa en navegador al endpoint propio GET /api/v1/soap/country/?code=MX. Django actua como cliente SOAP ante *CountryInfoService* consumiendo su WSDL con la libreria zeep y devolviendo la respuesta XML transformada en un JSON estructurado con nombre, capital, moneda y telefono de codigo de pais.

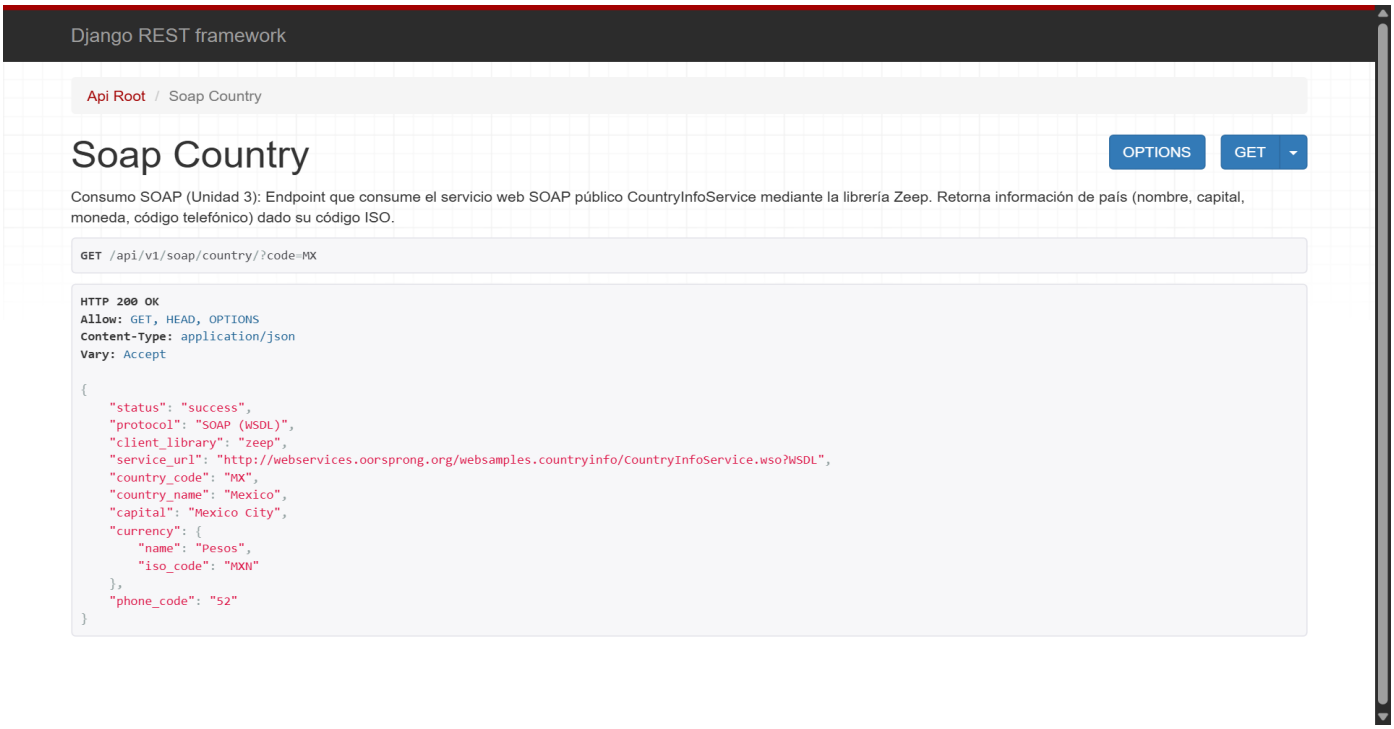


Figura: Evidencia 5: Consumo de Servicio Web SOAP Publico con Zeep (ra2_05_consumo_soap.png)

4. Instrucciones de Instalacion y Ejecucion Local

Para desplegar y verificar el sistema completo de forma local, seguir los pasos descritos a continuacion:

1. Clonar el Repositorio:
git clone https://github.com/Zerkantz/Django.git
cd Django

2. Configurar y Ejecutar el Backend (Django):
python -m venv venv
.\venv\Scripts\activate # En Windows
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
El backend quedara escuchando en http://127.0.0.1:8000/

3. Configurar y Ejecutar el Frontend (React + Vite):
cd frontend
npm install
npm run dev
El frontend estara disponible en http://127.0.0.1:5173/

4. Ejecutar la Bateria de Pruebas Automatizadas:
python manage.py test users
Garantiza que los 18 casos de prueba (seguridad, JWT, CRUD, SOAP y registro) se ejecuten con 100% de aprobacion.

5. Conclusiones y Cumplimiento de la Rubrica

Se cumplio a cabalidad con todos los criterios de evaluacion definidos para el **Resultado de Aprendizaje 2 y 3**:
- **Frontend React Vite:** Estructurado con layouts desacoplados, rutas protegidas, Formik, Yup y estilos Semantic UI/SASS.
- **Seguridad y Persistencia JWT:** Integracion con SimpleJWT a traves de AuthContext y almacenamiento local validado.
- **Consumo de APIs de Terceros:** Widget responsivo integrado en el Dashboard con peticiones asincronas fetch().
- **Web Services REST y SOAP:** Endpoints protegidos REST (/api/v1/auth/me/) y consumo SOAP empresarial con zeep.
- **Calidad de Codigo y Documentacion:** Cobertura de tests unitarios/integrales (18/18 OK), documentacion exhaustiva en README.md y sincronizacion con el repositorio oficial.